

LEARNING IN PUBLIC

Notes as We Learn — AWS ML

A living notebook for the AWS ML Engineer Associate (MLA-C01), filling in as the learning happens.

AWS CERTIFICATION · MLA-C01 · COMPILED ON PORTFOLIO-V2

0

Foundations — Math, Programming & “What Is a Model?”

The floor everything else rests on: the math, data, and programming intuition an ML engineer keeps reaching back to.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Math for ML: vectors, matrices & dot products → features and weights.
- Probability & statistics: distributions, mean/variance, conditional probability.
- Calculus intuition: derivatives & gradients → how models “learn”.
- Programming & data: Python, data structures, JSON; tabular data (rows = observations, columns = features).
- What a “model” really is: a function mapping features → prediction, tuned by data.

1

Core Machine Learning Concepts

The vocabulary that the whole certification is built on.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Feature vs. label; ML as a subset of AI that learns patterns without explicit rules.
- The three learning paradigms: supervised, unsupervised, reinforcement.
- Classification vs. regression; clustering, dimensionality reduction, anomaly detection.
- Overfitting vs. underfitting; the train / validate / test split.

2

Deep Learning & Modern AI

From artificial neurons to the foundation models behind generative AI.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Deep learning as a subset of ML; artificial neural networks and hidden layers.
- Automatic feature extraction (input → edges → parts → object).
- Generative AI: creating new text, image, audio, video, and code.
- Foundation Models and Large Language Models: pre-train once, adapt to many tasks.

3

Responsible AI, Problem Formulation & SageMaker

Turning a business problem into a well-posed ML problem — responsibly — and the platform it runs on.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Responsible AI — the eight dimensions (fairness, explainability, privacy & security, safety, controllability, veracity & robustness, governance, transparency).
- Problem formulation: define the business problem, choose the data, set measurable ML success criteria.
- Feasibility = $f(\text{data availability, problem complexity})$.
- Amazon SageMaker & SageMaker Studio: the managed build / train / deploy platform.

4

Domain 1 — Data Preparation

Collecting, ingesting and storing data, then transforming and validating it — the front door of the whole ML process and 28% of the MLA-Co1 exam.

Fundamentals of Data Collection

Why data collection matters

Think of data as the raw ingredients for cooking. A great chef cannot make a good meal from spoiled or wrong ingredients — and a machine learning model cannot make good predictions from poor data. The data you collect is the foundation the model learns from, so it decides how good the model can ever be.

A quick note on terms used throughout: a **machine learning (ML) model** is just a program that learns patterns from examples instead of being given fixed rules; **training** means showing the model those examples so it can learn.

What good collected data looks like: - It is **relevant** — it actually relates to the question you are trying to answer. - It has **enough quantity and enough variety** to learn from — but piling on more data does not help if that data is low quality. - It is **accurate and clean** — free of mistakes and unfair skews. - It is gathered **ethically and lawfully**, hiding personal identities when needed. - Bottom line: the *right* data produces an accurate model that copes with many real-world situations.

Centralize first, then prepare

Real-life picture: imagine a shop owner whose sales records are spread across a cash till, a paper notebook, some emails, and a phone app. Before spotting any trend, they first copy everything into one ledger. Machine learning data works the same way — it starts scattered across many places, so you bring it together before you can learn from it.

The order of work: 1. **Find** where all the data lives. 2. **Pick one central place** to keep it. 3. **Bring the data in** to that place. 4. **Clean, prepare, and secure** it so it is ready to learn from.

Common places data is stored

Before machine learning uses data, it usually sits in one of three kinds of storage. The easiest way to tell them apart is by what they hold and what job they are built for.

STORAGE TYPE	WHAT IT HOLDS	WHAT IT IS BUILT FOR	EVERYDAY WAY TO PICTURE IT
Data lake	Every kind of data, kept raw — neat table data, loose data like photos and text, and in-between data — pulled from many sources (other data-bases, sensors, live feeds, purchases)	A single raw collection point that later feeds analysis and models	A large storage yard where everything is dropped as-is, sorted later
Data warehouse	Only neat, table-shaped data, kept in related tables	Analysis and reporting. It is tuned for <i>Online Analytical Processing (OLAP)</i> — a fancy way of saying "quickly summarize huge amounts of data to answer business questions"	A well-organized library built for looking things up fast
Database	Neat, table-shaped data, smaller in scale	Day-to-day running of an app. It is tuned for <i>Online Transaction Processing (OLTP)</i> — handling many small, live reads and writes like recording a sale the moment it happens	A shop's live cash register and record book

A couple of terms in that table, in plain words: - **Structured data** = data that fits neatly into rows and columns, like a spreadsheet. **Unstructured data** = things with no fixed shape, like photos, video, or free text. **Semi-structured data** sits in between (more on all three in the next lesson). - Data warehouses are usually queried with **SQL (Structured Query Language)** — a standard language for asking a database questions like "how many customers cancelled last month?"

The one-line distinction to remember: a **data warehouse is for analysis**, a **database is for live day-to-day transactions**, and a **data lake holds every kind of data in raw form**.

What makes data "high quality" — four traits

Running example: a **fitness company trying to predict which members will cancel their membership** (cancelling is often called *churn*). If the data is poor or incomplete, the model struggles to find patterns and its predictions are unreliable. Good data has four traits:

- **Representative** — it mirrors the real world. If about 20% of members actually cancel each year, the data should show roughly that 20%. If it does not, the model will over- or under-predict cancellations.
- **Relevant** — it relates to the exact problem. To predict cancellations you want things like past visits and payment history, not unrelated trivia.

- **Feature-rich** — it includes enough useful details (each detail is called a **feature**) for the model to spot patterns. Too few details — say, leaving out member age or location — and the model misses real signals.
- **Consistent** — everything is recorded the same way. If one source writes dates as "1/11/22" and another as "Jan 11 2022", the mismatch confuses the model and lowers accuracy.

The takeaway: **quality of data in = quality of results out**. Better data leads to better machine learning results.

Coming up next

The next lesson looks closely at the *types* of data — and how traits like accuracy, completeness, and relevance shape how well a model can learn.

Concepts explained simply

Who does all this data work? (it is a team sport)

Preparing data — collecting it, cleaning it, shaping it, storing it — is rarely one person's job. The same dataset flows through several pairs of hands: a **data engineer** gathers and re-shapes raw data into usable form, a **data scientist** experiments with it to find patterns and build models, a **machine learning engineer** turns a working experiment into dependable production software, and an **operations-focused engineer** owns the automation that keeps it all running. Amazon Web Services (AWS) treats these role names as a loose organizing device, not fixed job titles — what the exam really tests is *which tool fits which need*. AWS groups its data-preparation tooling by **use case**: a visual point-and-click path for non-coders, a code-and-SQL (Structured Query Language) path for analysts, and a heavy-duty path for data too big for one machine. The glue that makes preparation genuinely *shared* is a **feature store** — a central, searchable pantry of already-prepared data ingredients that every team member can discover and reuse instead of re-cooking them from scratch. **In real life:** Think of a restaurant kitchen. One person sources and stocks the pantry, the chef invents the recipe, the line manager turns the winning recipe into a repeatable process for every shift, and the labeled pantry of prepped ingredients is what lets them all work from the same stock without duplicating effort. **Why it matters:** Reusing centrally-prepared data avoids repeated work and, importantly, avoids the subtle bug where a model is trained on data prepared one way but makes live predictions on data prepared another way.

EXAM TRAP

"Shared responsibility" here means shared across *human roles in the team*. Do not confuse it with the AWS Shared Responsibility Model, which is a completely different security concept about what AWS secures versus what the customer secures.

Why data preparation decides everything (garbage in, garbage out)

A model learns *only* from the data you feed it, so data quality sets a hard ceiling on how good the model can ever be. Feed it wrong, skewed, incomplete, or inconsistently-recorded data (the same country written as "United States" in one file and "US" in another) and it faithfully learns the wrong patterns. AWS describes the full machine learning journey in six phases, in order: agree on the business goal, frame it as a learning problem, process the data, develop the model, deploy it, and monitor it. Two things to remember about that journey: **data processing sits near the front**, so any defect there flows downstream into everything else — and the journey is a **loop, not a straight line**: monitoring the live model catches the world drifting away from the training data and feeds fresh data back into re-training. **In real life**: Building on a bad foundation — every floor above inherits the flaw, and the earlier you catch it, the cheaper it is to fix. **Why it matters**: Collecting, ingesting, and storing data well is the front door of the whole process. It is the cheapest place to fix problems and the most expensive place to ignore them.

Types of Data

Why the type of data matters

The kind of data you have strongly shapes which learning method works best — a bit like how a recipe depends on the ingredient. Plain numbers in a table suit simple prediction methods, while photos usually need more powerful, brain-inspired methods. Preparing and reshaping the data first almost always improves the result.

There are really **two separate questions** you can ask about any dataset. Learners often mix them up, so keep them apart: - **Question 1 — What kind of content is it?** (This picks the learning method.) - **Question 2 — How organized is it?** (This tells you how much cleanup it needs.)

Question 1 — What kind of content is it?

Data usually falls into one of four kinds:

KIND	WHAT IT IS	EVERYDAY EXAMPLE	TYPICALLY USED FOR
Text	Words and documents	product reviews, emails	understanding language, such as judging if a review is positive or negative
Tabular	Numbers arranged in rows and columns, like a spreadsheet	a sales table	straightforward predictions from neat numbers
Time series	Values recorded over time, in order	daily temperatures, stock prices	spotting trends and forecasting what comes next
Image	Pictures made of coloured dots (pixels)	photos, scans, video frames	recognizing what is in a picture

{{img: content-types-flow.png | The four kinds of content you will meet: text, tabular, time series, and image.}}

Question 2 — How organized is it?

This is about the *shape* the data comes in, which decides how much tidying it needs before a model can use it. Three levels:

LEVEL	PLAIN MEANING	NEEDS HOW MUCH CLEANUP?	EVERYDAY EXAMPLES
Well organized	Already in neat rows and columns with a fixed layout	Very little	Parquet and ORC files (compact table-style files built for fast analysis)
Partly organized	Has helpful labels or tags, but not a strict table	Some	CSV files, spreadsheet exports, and JSON files (a common text format that stores data as labelled fields)
Not organized	No fixed shape at all	The most — but it holds the richest detail	plain text, photos, video

The three levels look like this — from tidy grids to loose, mixed material:

{{img: structured.png | Well organized: neat, uniform rows.}} {{img: semi-structured.png | Partly organized: labelled, but mixed.}} {{img: unstructured.png | Not organized: free-form text, images, video.}}

The most common mix-up to remember: a **CSV file looks like a neat table, but it counts as partly organized, not well organized.** (CSV stands for *comma-separated values* — a plain text file where each value is separated by a comma.)

Putting the two questions together

The same piece of data answers *both* questions independently. A photo is "image" content (Question 1) **and** "not organized" in shape (Question 2). A sales spreadsheet is "tabular" content **and** "partly organized" as a CSV file. That is the whole point — they are two different lenses, not one.

This is exactly why the previous lesson insisted on gathering everything into a **data lake** first: a lake happily holds all three levels of organization side by side, which is what lets you keep the messy-but-rich material instead of throwing it away. The catch is that a model cannot learn from raw, not-organized data directly — so the very next step in the journey, **transforming the data**, is about reshaping these loose formats into something a model can actually use. Types and organization are the vocabulary you will lean on the moment that transformation work begins.

Concepts explained simply

How can a computer tell if a written review is happy or unhappy?

Natural Language Processing (NLP) means teaching computers to read and make sense of everyday human language, the kind you speak and type, such as emails, reviews, or tweets. Normally computers only understand numbers, not words, so NLP is the bridge that lets a machine work with writing. One popular NLP job is figuring out the feeling behind a piece of text, whether it sounds positive (happy or approving), negative (unhappy or complaining), or neutral (no strong feeling either way). The tricky part for beginners: a computer cannot do math on the word "great" or "terrible" directly, so before any learning can happen, the text must first be turned into numbers. Turning words into lists of numbers is the required first step, because a computer learns only by crunching numbers, not letters. Once the words are numbers, the machine can spot patterns and predict the feeling. There is also a ready-made cloud tool from Amazon Web Services (AWS) that does all of this for you: you hand it text, and it labels the overall feeling as Positive, Negative, Neutral (no strong feeling), or Mixed (contains both happy and unhappy parts), and it gives a certainty number showing how sure it is, close to 100 percent when very sure and close to 0 when barely sure. **In real life:** A pizza restaurant gets 500 online reviews overnight. Instead of a person reading each one, they feed the reviews into one of these tools. It reads "The crust was amazing and the staff was friendly!" and labels it Positive, while "Cold food and rude service, never again" gets labeled Negative. In seconds the owner sees that 80 out of every 100 reviews are Positive, without reading a single one by hand, like having an assistant who instantly sorts every comment into a happy pile or an unhappy pile. **Why it matters:** Businesses use this to automatically watch customer opinions at huge scale, for example scanning product reviews, support messages, or social media to catch complaints early. When you want this done without building and training your own program, the ready-made AWS text-reading service is the standard choice.

Why is tabular data a good fit for predicting a number with linear regression?

Linear regression is a method that predicts a number by learning how it relates to other numbers. It studies examples and works out a straight-line rule, meaning a simple pattern like "the answer goes up by a fixed amount every time an input goes up by one." Once it learns that rule, you feed it new inputs and it gives you a predicted number, and that number can be any value, including decimals like 3.7, not just whole categories. Tabular data just means data arranged in a neat table of rows and columns, like a spreadsheet, where each row is one example (say, one house) and each column is one measured number (size, number of bedrooms, price). This fits the method perfectly because it needs exactly that shape: each column that describes something becomes an input the line uses, and the one column you want to predict becomes the answer it aims for. So the reason a table is a good fit is that its layout already matches what the math needs. Every input the rule multiplies and adds up is sitting in its own column, and every row is a ready-made example to learn from, so nothing needs rearranging. **In real life:** Picture a spreadsheet of houses. Each row is one house; the columns are its size in square feet and its selling price. Linear regression studies these rows and finds a rule like "price = 100,000 + 150 times the size."

Then for a new house where you only know the size, you plug the size into that rule and it predicts the price. The size column is the input, the price column is the answer being learned, and each row is one lesson the method learns from. **Why it matters:** Most business and real-world data already lives in tables (sales logs, sensor readings, customer records), so linear regression can be applied straight away to predict a numeric outcome like price, demand, or temperature. This is why table-shaped number data is the go-to answer for linear regression, and why the matching AWS tool for this method expects data laid out exactly this way, with rows as examples and columns as the measured facts.

Data Visualization & Exploratory Data Analysis

What this lesson is about, in one line

Before you train a model, you *look at* your data with charts to understand it and catch problems early. Exploring data with charts and summaries is called **Exploratory Data Analysis (EDA)**.

Why look at data first (data visualization)

Drawing your data as pictures helps you judge its quality and relevance *before* training. By seeing the data, you can spot problems like odd values, lopsided spreads, and outliers (values far from the rest). Catching these early means the model is less likely to end up inaccurate.

Three goals of looking at your data: 1. **Understand the data** — see its patterns, trends, shape, and limits. This helps you pick the right method. 2. **Identify quality issues** — find missing values, outliers, and inconsistencies. Clean data is essential. 3. **Shape the data** — improve it through changes and by creating better inputs, so it is ready for modeling.

Four ways charts help you explore (EDA methods)

- **Relationship analysis** — do two things move together? (helps pick useful inputs and drop repetitive ones)
- **Distribution analysis** — where do values cluster, and how spread out are they? (shows the typical value, the variety, the shape, and outliers)
- **Comparison** — how do groups differ? (reveals separate trends between subgroups)
- **Composition** — what are the proportions? (shows percentages and counts that make up a whole)

Two broad kinds of data → different charts

- **Categorical data** = labels/groups, not amounts. Examples: gender, membership tier, dietary preference. You summarise it with counts and percentages.
- **Numerical data** = measured amounts. Examples: age, income, test score. It supports more math (averages, spreads).

Charts for CATEGORICAL data

Categorical data has a **limited number of possible values with no inherent order**, so its analysis leans on **counts, percentages (proportions), and cross-tabulations** (tables counting how often two categories occur together).

{{img: 01-categorical-charts.png | The three go-to charts for categorical data: bar chart, pie chart, and heatmap.}}

CHART	WHAT IT SHOWS	EDA METHOD IT ASSISTS
Bar chart	Bars sized by how much of the data falls in each group	Comparison
Pie chart	A circle representing the whole dataset, split into slices — each slice is one category's share	Composition
Heat-map	A grid that uses colour to show values and patterns	Relationship

Bar chart, worked example — catching skew: chart the education levels *in your dataset* next to the same chart for the *general population*. If high-school diplomas are under-represented and advanced degrees over-represented in yours, your data is **skewed** — and skewed data can bias the model. This is quality-checking with a single picture.

{{img: 02-education-barchart.png | Dataset vs general population: the mismatch in bar heights reveals under- and over-represented education levels — a skew warning.}}

Pie chart: a quick visual summary of how a categorical column is distributed — every slice is a percentage of the whole.

{{img: 03-pie-dataset.png | The same education data as a composition: each slice is one category's share of the whole dataset.}}

Heatmap: colour makes clusters and patterns jump out far faster than scanning a table of numbers — for example, strong positive links in green and strong negative links in blue. One caution from the course: the colours only *point* at a pattern; to understand it you still have to read the actual numbers behind them.

{{img: 04-correlation-heatmap.png | A heatmap of how strongly each pair of columns moves together — green for strongly positive, blue for strongly negative.}}

Charts for NUMERICAL data

Numerical data supports more advanced math, so its charts lean on statistics like the **mean** (average), **median** (middle value), and **standard deviation** (typical distance from the average).

{{img: 05-numerical-charts.png | The numerical toolkit: box plot, histogram, and scatter plot.}}

CHART	WHAT IT SHOWS	EDA METHOD IT ASSISTS
Scatter plot	Each point placed by two values (an x and a y)	Relationship
Histogram	Bars counting how many values fall in each range (bin)	Distribution
Density plot	A smooth curve of where one feature's values concentrate — no fixed bins	Distribution
Box plot	The median, the middle 50% as a box, whiskers to the min/max, and outliers as separate dots	Distribution (and comparing groups' spreads)

Scatter plot, worked example: a medical dataset of tumour measurements, where each case is either malignant or benign. Plot the cases with two different markers (say triangles vs circles) and look for **distinct regions** — if the two classes cluster in different areas of the plot, the two measured features genuinely separate the classes, which is exactly what a model can learn from.

{{img: 06-scatter-breastcancer.png | Malignant (triangles) vs benign (circles): the two classes occupy visibly different regions of the scatter plot.}}

Histogram, worked example: plot incomes into ranges (\$0–100K, \$100–200K, ...) and the bars answer three questions at a glance: is the data **normally distributed** (one symmetric hump)? **how many peaks** are there? is there **skewness** (a lopsided tail)?

{{img: 07-histogram-income.png | Income binned into ranges — the bar shape reveals the distribution, its peaks, and any skew.}}

Density plot: the histogram's smooth cousin — same question (where do values concentrate?) without forcing the data into bins. A sharp peak shows exactly where values pile up.

{{img: 08-density-plot.png | A density plot: the peak marks where the values concentrate.}}

Box plot — how to read one: the box's *position and length* show where the bulk of the data sits and how spread out it is (a short box = tightly clustered values). The line inside the box is the **median** — if it sits off-centre, the data is skewed. The **whiskers** span the range excluding outliers, and **outliers** are drawn as individual points beyond them. Box plots shine when comparing the spreads of several groups side by side.

{{img: 09-boxplot.png | Anatomy of a box plot: box = middle half of the data, centre line = median, whiskers = range, dots = outliers.}}

KEY IDEA

match the chart to the question — comparing groups (bar), proportions of a whole (pie), colour-coded patterns across two dimensions (heatmap), two amounts together (scatter), the shape of one amount (histogram or density), spread and outliers (box).

EXAM TRAP

"find hidden patterns and relationships" in a dataset points to the **relationship-analysis** charts — the **heatmap** (categorical/cross-feature patterns via colour) or the **scatter plot** (two numerical variables) — not a histogram or density plot, which describe the distribution of a *single* feature.

Furnished images for this node (files: `images/M1_1c/`)

Inline `{{img:}}` markers above pull from `images/M1_1c/` per MANIFEST.md filenames (01–09). Drop the image files there and each figure appears at its concept; missing files are silently skipped.

AWS Storage Options (Amazon S3, EBS, EFS, FSx)

What this lesson is about, in one line

Once data is collected and analyzed, you pick *where it lives* on AWS — and the right choice depends on your use case's cost, performance, data structure, and access pattern.

The four storage services

Amazon Web Services (AWS) offers four storage services commonly used for machine learning tasks that need scalable, durable, highly available storage: - **Amazon Simple Storage Service (Amazon S3)** — object storage - **Amazon Elastic Block Store (Amazon EBS)** — block storage attached to compute - **Amazon Elastic File System (Amazon EFS)** — shared file storage that grows on its own - **Amazon FSx** — managed versions of popular specialist file systems

Each excels at something different; no single one wins every use case.

Amazon S3 — the flexible object store (and your data lake)

Amazon S3 is a flexible, scalable **object storage** service used for data lakes, websites, cloud apps, backups, archives, analytics, and machine learning. In ML workflows it usually plays the role of the **central data lake**: the one place data is ingested into, extracted from, and transformed through on its way to other AWS services.

`{{img: 01-s3-benefits.png | Amazon S3 key benefits: API access, streaming/copying between buckets, built-in redundancy, and cost-optimizing storage classes.}}`

Key benefits: - Highly scalable, available, and redundant; accessed through an **API** (a programmatic interface — no drive letters or mounts) - Data can be **streamed or copied** between S3 buckets and other services - **Storage classes** cut costs by matching how often data is accessed — frequent use, infrequent access, archive, or automatic tiering

WATCH OUT

S3's one trade-off is **higher latency** than local storage — it is reached over the network. For latency-sensitive workloads it may not be optimal; with caching and good architecture most applications still perform excellently, but you must account for its network-based nature.

Machine learning use cases: - **Data ingestion and storage** — large training datasets land in S3 by streaming or batch, then feed training and inference. - **Model training and evaluation** — S3 stores datasets *and* trained models, and its **versioning** lets you keep and compare model iterations to evaluate performance. - **Integration hub** — Amazon SageMaker trains and deploys straight from S3 data, Amazon Kinesis streams data into S3 buckets, and AWS Glue connects to S3 for data processing.

Amazon EBS — fast disks for one machine

Amazon EBS provides **persistent, block-level storage volumes** (virtual disks) that attach to Amazon Elastic Compute Cloud (Amazon EC2) instances, so you can scale storage performance and cost as needed. Well-suited for databases, web applications, analytics, and ML; it integrates with Amazon SageMaker as a core component for model training and deployment.

Key benefits: - **High-performance, low-latency** block storage attached directly to an EC2 instance - Choice of **hard disk drive (HDD)** volumes for frequent low-cost storage or **solid state drive (SSD)** volumes for workloads needing high **IOPS** (input/output operations per second — how many reads/writes per second the disk sustains) - **Point-in-time snapshots** that can be restored — disaster recovery and data protection built in

WATCH OUT

EBS keeps storage *separate* from the EC2 instance, which adds planning: volumes must be allocated and scaled per instance. The alternative, an **instance store**, ties storage directly to the instance's lifecycle — simpler to manage, but it lives and dies with the machine. EBS trades a little management for flexibility and persistence.

Machine learning use cases: - **High-performance storage** — high-IOPS volumes give ML applications fast reads/writes on large datasets, accelerating workflows. - **Hosting pre-trained models** — upload, store, and serve pre-trained models for real-time predictions without separate hosting infrastructure.

Amazon EFS — one shared folder for many machines

Amazon EFS is a highly scalable, **serverless file storage** service: you get file systems that grow and shrink automatically as files are added or removed — up to **petabytes** — with no capacity to provision or manage, and performance stays high as usage changes. It speaks the standard **NFSv4** protocol (Network File System — the long-standing way computers

share folders over a network), so existing applications built for on-premises NFS servers migrate **without code changes**.

{{img: 02-efs-benefits.png | Amazon EFS flashcards: scalable and performant, high throughput and parallelism, low-latency access at petabyte scale.}}

Key benefits: - **Scalable and performant** — a high-performance file system accessed **concurrently from multiple EC2 instances** - **High throughput** — supports parallel access for data processing, media processing, and content management - **Low-latency access** — petabyte-scale shared file systems without performance issues

WATCH OUT

EFS costs more per gigabyte than EBS. The trade: EFS buys effortless *shared* scaling; EBS is cheaper but single-instance and workload-dependent. Decide whether streamlined shared file systems justify the price.

Machine learning use cases: - **Concurrent access** — many EC2 instances read the same datasets simultaneously, ideal for ML workflows sharing data across compute. - **Shared datasets** — no copying large datasets to every instance; libraries, frameworks, and models are read by all instances at once without contention, speeding training and deployment.

Amazon FSx — managed specialist file systems

Amazon FSx is a fully managed service providing **popular specialist file systems**: **Lustre** (high-performance computing), **NetApp ONTAP**, **OpenZFS**, and **Windows File Server**. It focuses on reliability, security, and scalability for ML, analytics, and high-performance computing — delivering **millions of IOPS with sub-millisecond latency**.

{{img: 03-fsx-benefits.png | Amazon FSx key benefits: shared mounts, high performance, four file-system types, ephemeral and persistent storage options.}}

Key benefits: - **Shared access** — mount one file system to many compute instances - **High performance** — low latency, high throughput - **File system variety** — Lustre, NetApp ONTAP, OpenZFS, Windows File Server - **Storage options** — ephemeral (temporary, for short-lived tasks) or persistent (long-term retention)

WATCH OUT

specialist file systems add complexity and management overhead, and tightly coupling your ML workflow to one file system risks **vendor lock-in**, limiting future flexibility.

Machine learning use cases: - **Lustre for training** — Lustre's **distributed architecture** gives highly parallel, scalable data access, ideal for large, high-throughput ML training datasets. - **Managed operations** — backups, scaling, high availability, and security are handled for you; you focus on data and applications, not infrastructure.

Choosing between them — the decision table

SERVICE	STORAGE KIND	SHINES WHEN	WATCH FOR
Amazon S3	Object (API access)	Central data lake; huge, durable, cheap; versioned models	Network latency
Amazon EBS	Block (one instance)	Fast disks for a single machine; high IOPS; snapshots	Per-instance planning; not shared
Amazon EFS	File (shared, NFS)	Many Linux instances reading the same files, auto-scaling to petabytes	Higher cost than EBS
Amazon FSx	File (specialist)	Windows File Server needs, or Lustre for extreme-throughput training	Complexity; vendor lock-in

EXAM TRAP

shared-access questions hinge on the *file system named in the scenario*. "Concurrent access from multiple instances **to a Windows File Server**" (or extreme throughput with sub-millisecond latency via Lustre) → **Amazon FSx**. Generic shared Linux file access over NFS → **Amazon EFS**. One instance needing fast disks → **Amazon EBS**. Cheap, durable, latency-tolerant bulk storage → **Amazon S3**.

Concepts explained simply

Object vs block vs file storage — what is actually different?

Think of three ways to keep your belongings. **Object storage** (Amazon S3) is a valet warehouse: you hand over an item with a claim ticket and get the whole item back on request — you never see shelves, and the warehouse can be effectively infinite, but every retrieval is a round trip to the warehouse. **Block storage** (Amazon EBS) is the hard drive bolted into your own computer: the machine can read and write any tiny piece of it instantly, which is fastest of all, but it belongs to that one machine. **File storage** (Amazon EFS and Amazon FSx) is the shared network drive at an office: everyone sees the same folders at once and works on the same files without making copies. **In real life:** a photo studio might archive every shoot in the valet warehouse (cheap, unlimited), edit today's shoot on the workstation's own drive (fastest), and keep fonts and templates on the shared office drive so every editor uses the same ones. **Why it matters:** most exam storage questions are really asking "which of the three kinds fits this access pattern?" — one machine (block), many machines sharing (file), or anything-at-scale over the network (object).

Furnished images for this node (files: [images/M1_1d/](#))

Inline `{{img:}}` markers above pull from [images/M1_1d/](#) per MANIFEST.md filenames. Drop the image files there and each figure appears at its concept; missing files are silently skipped.

Data Formats & File Types (row · column · object notation)

Why format matters

Common data formats — **row-based**, **column-based**, **object-notation** — structure data for ML. Format **impacts efficiency of analysis** and should **match the data structure and operations needed**.

1. Row-based data format

Each **row** = **one record/entity**; columns = that entity's features. Common in relational databases & spreadsheets; shows relationships between features. - **Good for:** dense, tabular data **frequently accessed by rows**. - Example: a customer table (customerID, name, age, email, last_support, subscription_active) → analyze relationships like subscription status vs age.

Row-based file types: | Format | Notes | |---|---| | **CSV** (comma-separated values) | A lightweight text file where each line is one row and commas separate the values. Stores a mix of words and numbers, so it is very common. **Trade-off:** it is simple, but slower for heavy analysis than the column-first files described below. | | **Avro RecordIO** | Stores records one after another (row by row), which helps when a model reads through the whole dataset many times while training. It also carries a built-in description of its own layout (a **schema**), so programs read it faster than files with no fixed layout. |

2. Column-first data format

Stores data with **columns as the primary structure**; queries extract insights from **patterns within a column** rather than whole records → efficient trend analysis across large datasets. - **Good for:** **sparse** data with aggregations done by column. - Example: analyze churn across many records (last_support, subscription_active) without loading full records. Column-first → **compression** + faster access for ML.

Column-first file types: | Format | Notes | |---|---| | **Parquet** | Column-first; used in analytics/data-warehouse workloads with large datasets. ML benefits from **compression** → better storage + performance. | | **ORC** (Optimized Row Columnar) | Column-first, similar to Parquet; used in big-data (**Apache Hive, Spark**). Efficient compression + performance → widely chosen for ML. |

3. Object-notation data

Fits **non-tabular, hierarchical** data (graphs, textual). Structured into hierarchical **objects with features & key-value pairs** rather than rows/columns.

Object-notation file types: | Format | Notes | |---|---| | **JSON** | Document-based; human + machine readable. **Flexible, compact, hierarchical, easy to parse**. Built from **objects** (key-value pairs in `{ }`; values = string/number/Boolean/array/object/null) and **arrays** (values in `[ ]`, comma-separated). | | **JSONL** (newline-delimited JSON) | JSON objects separated by **new lines**, not nested. Each object on its own line. **Efficiency:** pro-

cess individual objects **without loading a whole JSON array** → better for large ML datasets. Can **map to column-first formats like Parquet** for those added benefits. |

Exam-relevant summary

- **Row-based** (CSV, Avro RecordIO) → dense, row-access, full-dataset iteration.
- **Column-first** (Parquet, ORC) → sparse, column aggregation, compression, analytics — **preferred for large ML/analytics workloads**.
- **Object-notation** (JSON, JSONL) → hierarchical/flexible; JSONL scales better than JSON arrays and can map to Parquet.
- Ties to **M1_1b**: Parquet/ORC = structured; CSV/JSON = semi-structured. Format choice = evaluate algorithm, use case, data, analysis method.

Concepts explained simply

What makes Parquet and ORC files so much faster for big data?

Parquet and ORC (Optimized Row Columnar) are two ways of saving big tables of data on disk, think giant spreadsheets with rows and columns. Both are free and open (anyone can use them). The special thing is HOW they physically store the data. Most everyday files, like a CSV (Comma-Separated Values) file, which is a plain-text table where columns are separated by commas, are "row-first": they write out the whole first row, then the whole second row, and so on. Parquet and ORC are "column-first": they keep all the values from one column together, then all the values from the next column together. Why does that matter? Because most analysis and machine learning only needs a few columns out of many (for example, just "age" and "income" out of 50 columns). With a column-first layout the computer can jump straight to just those columns and skip the rest, so it reads far less data and finishes much faster. These formats also shrink smaller (called compression, which means packing a file down so it takes up less space): since all the values in one column are the same kind of thing (all dates, or all prices), they look alike and can be packed tightly, saving storage space and money. **In real life:** Imagine a school with 10,000 students, and you only want the average height. In a row-first setup, each student's full record (name, address, grades, height, and so on) is stored together, so to get heights you must flip through all 10,000 complete records and pull one number from each. In a column-first setup (Parquet or ORC), every student's height is already stored side-by-side in one place, so you grab just that single list and instantly average it, without ever touching names, addresses, or grades. **Why it matters:** Machine learning and data analysis love these formats because they usually read a few columns across millions of rows. On AWS, several tools (including one that answers questions about files using plain database-style requests, and others that prepare and train on data) read Parquet and ORC much faster and cheaper than row-first CSV files, because they pull far less data out of Amazon Simple Storage Service (S3), Amazon's file storage where you pay based on how much data is read.

How does storing data one full row at a time reveal patterns like "older customers cancel less"?

A "row-based format" means you store your data like a table where each row is one thing (for example, one customer), and the columns are facts about that thing (like their age, how long they have been a customer, and whether they cancelled). Because everything about ONE customer sits together on ONE row, all their facts are lined up side by side. A "relationship" between two of those facts just means: when this fact changes, does that other fact tend to change too? You spot it by scanning down the table and comparing two columns row by row. For example, if you look at the age column next to the cancelled column and notice that the older-age rows almost always say "No" for cancelled, you have found a relationship: older customers cancel less. The row layout makes this easy because the age and the cancellation answer for the same person are already glued together on the same line, so you never have to hunt around to figure out which age goes with which cancellation. **In real life:** Picture a class attendance sheet where each row is one student. Columns: Age, Hours Studied, Passed?

STUDENT	AGE	HOURS STUDIED	PASSED?
Ana	25	10	Yes
Ben	19	2	No
Carla	31	12	Yes
Dan	18	1	No

Reading down the rows, the "Passed? = Yes" people all sit next to bigger "Hours Studied" numbers, and the "No" people next to tiny ones. Because each student's hours and result share the same row, you instantly see the pattern: more study hours goes with passing. Same idea as "older customers cancel less", where each customer's age sits right beside their cancellation answer, so the pattern jumps out. **Why it matters:** Most machine learning starts with a plain table (like a spreadsheet or CSV file, a simple text table), where one row equals one example the program learns from. Keeping each thing's facts together on one row is exactly what lets both a person (while exploring the data by hand) and a learning program line up an input (age) with an outcome (cancelled) for the same customer, which is how the program learns "when age is high, cancellation is low." This row-by-row storage is also the natural fit for everyday systems that read or write one whole record at a time, such as processing a single order.

Why do training tools like files that store examples in order with a built-in description?

Think of your training data as a long list of examples, where each example is one thing the program learns from (like one customer, one photo, or one row of numbers). Avro is simply a way of packing this data into a single file. Two ideas matter. First, the examples are written into the file one right after another, like beads on a string, so a training program can start at the front and read straight through to the end in order. That is fast and smooth, which helps because training usually reads the whole set of data over and over. (The general name for this style of file that packs many examples back-to-back is RecordIO; Avro is one specific format that does this and adds the second idea below.) Second, the file carries a

tiny built-in description of what each example looks like: the field names (like "age", "price", "answer") and the kind of value each field holds (number, text, or true/false). Because that description travels inside the file itself, any program opening it instantly knows how to read the data correctly instead of guessing. **In real life:** Imagine a filing cabinet where every folder is filled in on the same pre-printed form (Name, Date, Amount), and a cover sheet up front explains exactly what those blanks mean. Anyone can grab folders one after another and read them without confusion. That is this kind of file. The opposite is a shoebox of loose sticky notes in random handwriting with no key: you would have to guess what each scribble means, and there is no clean order to read them in. **Why it matters:** A file that describes itself and packs its examples in order lets machine learning training tools move through huge amounts of data quickly and reliably. On Amazon Web Services (AWS), a closely related packed format is the recommended, efficient input for many ready-made learning programs, and it works well with a setting that feeds data straight from storage to the training program on the fly, instead of copying all of it to the machine first.

How do Hive and Spark handle data too big for one computer?

Imagine you have so much data that no single computer can handle it, so you spread it across many computers working together (often called "big data"). Hive and Spark are two free, open tools that help you process this huge pile of data across all those machines. Hive lets you ask questions about giant sets of data using SQL (Structured Query Language), the simple, English-like way of asking a database questions, so you do not have to learn anything new. Spark is a fast, do-many-things worker that actually performs the heavy number-crunching, and it can also train machine learning programs (programs that learn patterns from data). The reason these come up alongside file formats like ORC (Optimized Row Columnar, a compact way of storing table data on disk so it takes less space and reads faster) is simple: these tools need to read and write the data somewhere, and ORC is one storage format they work with especially well. In short: ORC is how the data sits on disk, while Hive and Spark are the tools that read that data and do useful work with it. **In real life:** Picture a giant library with a billion books that one librarian could never sort alone. Instead, you hire a whole team and split the job: each worker takes one shelf and counts, sorts, or searches their section at the same time, then everyone combines their answers. Hive and Spark are like the manager who hands out these chunks to many computers at once. Hive is the manager you talk to in plain "please find all books published after 2000" style requests (SQL), while Spark is a faster, more versatile manager who can also teach the workers to spot patterns (machine learning). ORC is like the neatly labeled, space-saving shelving system the books sit in, so any worker can grab what they need quickly. **Why it matters:** Real machine learning starts with preparing enormous amounts of data. Hive and Spark let you clean, filter, and reshape data far too big for one computer, and Spark can even train programs at large scale. On AWS you meet them inside managed big-data services that set up and run these tools for you.

Data Ingestion: Batch vs Streaming (+ Amazon Kinesis)

Two ways to ingest into a centralized location

KEY FRAMING

Training is primarily done on **historical data processed in BATCH**; **inference/predictions** are primarily done with **STREAMING** data.

Batch data

Data collected & processed **in batches, not continuously**. Processed once or on a **recurring schedule**. Usually **not time-critical**; used for **long-term analysis**. - Batch processing analyzes the **entire dataset at once** to train ML models (vs sequential/incremental). - Common for **traditional ML** where the full dataset is available upfront. - **Example**: a data scientist has 1M images to train an image classifier — the whole dataset is processed in batches during training; the model iterates over the images many times, updating parameters to optimize performance.

Streaming data

Data **continuously generated & processed in real-time / near-real-time**. Can be batched, but streaming needs extra considerations: **data buffering, real-time monitoring, fault tolerance, scalability** to handle the constant flow. - Processes info **as it becomes available** → useful when **immediate action/predictions** are needed. - **Example**: a fraud-detection engine analyzes transactions **as they occur**. Real-time streaming apps often rely on models **trained on historical batch data** — combining live streaming + batch-derived insights → instant decisions as events unfold.

Amazon Kinesis (streaming ingestion family)

- **Kinesis Data Streams (KDS)** — real-time, low-latency, pipe whose capacity you control; you build the program that reads it; **replayable** retention (24h default → up to 365 days). Capacity is set in units you control.
- **Amazon Data Firehose** (formerly Kinesis Data Firehose) — fully managed, no capacity setup, auto-scales, **near-real-time**; auto-delivers to **S3, Redshift, OpenSearch, Splunk, Snowflake (native), Apache Iceberg**, HTTP endpoints; can transform via Lambda + convert JSON→Parquet/ORC.
- **Amazon Managed Service for Apache Flink** (formerly Kinesis Data Analytics) — process streams **in-flight** (SQL/Java/Python/Scala).
- **Kinesis Video Streams** — time-encoded video/media for playback + ML.

EXAM TRAP (THE CLASSIC)

Data Streams = real-time, you control capacity, replayable, sub-second → pick when *lowest latency*. **Firehose** = near-real-time (buffered), no capacity setup, auto-delivers to fixed destinations, no long-term replay → pick for *zero-admin delivery to S3/warehouse*.

Batch vs streaming — summary

	BATCH	STREAMING
Timing	Scheduled / one-time	Continuous, real-time
Time-critical?	No (long-term analysis)	Yes (immediate action)
ML use	Training (historical)	Inference/predictions
AWS	S3 + Glue/EMR	Kinesis family
Example	1M images → image classifier	Fraud detection on live transactions

STILL TO COME IN THIS CHAPTER

More of this chapter is on the roadmap and will fill in as the notes are written.

Planned topics:

- Data extraction (M1_1g) — awaiting course input.
- Data merging (M1_1h) — awaiting course input.
- Ingestion & storage troubleshooting (M1_1i) — awaiting course input.
- Transform Data (M1_2): cleaning, encoding, feature engineering; Feature Store, Data Wrangler, Glue.
- Validate & Prepare for Modeling (M1_3): bias mitigation, split/shuffle/augment; DataBrew, Data Quality.

5

Domain 2 — Model Development

Choosing an approach, training and tuning, then evaluating the result — 26% of the exam.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Choosing a modelling approach for the problem and data.
- Training and hyperparameter tuning.
- Model evaluation and metrics.

6

Domain 3 — Deployment & Orchestration

Getting a working model into dependable production — infrastructure, CI/CD and pipelines — 22% of the exam.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Deployment infrastructure and endpoints.
- CI/CD for ML.
- Orchestrating ML pipelines.

7

Domain 4 — Monitoring, Maintenance & Security

Keeping a live model healthy — drift, cost/performance and security — 24% of the exam.

COMING AS WE LEARN

This chapter is on the roadmap. It will fill in with worked notes as the learning gets there — for now, here is what it will cover:

- Monitoring for data and model drift.
- Cost and performance optimisation.
- Securing ML workloads.